Brilliant Mind Solutions LLP - Madhav

C# Intermediate to Tough Interview Preparation

Topic 1: Classes and Objects

1. What is the difference between a class and an object?

A class is a blueprint defining data and behavior. An object is an instance created from that class.

```
public class Student
{
    public string Name { get; set; } = string.Empty;
    public void Display() => Console.WriteLine($"Student: {Name}");
}

var student = new Student { Name = "Ravi" };
student.Display();
```

2. How are reference types stored and copied?

A class variable stores a reference to an object. Assigning it to another variable copies the reference, so both variables refer to the same object.

```
public class Employee
{
    public string Name { get; set; } = string.Empty;
}

var first = new Employee { Name = "Anita" };
var second = first;
second.Name = "Meena";
Console.WriteLine(first.Name); // Meena
```

3. What happens when an object is created using new?

Memory is allocated, fields receive default values, initializers run, base construction occurs, the current constructor runs, and a reference is returned.

```
public class Account
{
    private string _owner = "Unknown";
    public Account(string owner)
    {
        _owner = owner;
        Console.WriteLine("Constructor executed");
    }
}

var account = new Account("Lakshmi");
```

4. How do fields, properties, methods, and events differ?

A field stores data, a property controls access, a method defines behavior, and an event notifies other objects.

```
public class Order
{
    private decimal _total;
    public int Id { get; set; }
    public decimal Total { get => _total; private set => _total = value; }
    public event EventHandler<decimal>? AmountAdded;
    public void AddAmount(decimal amount)
    {
        if (amount <= 0) throw new ArgumentOutOfRangeException(nameof(amount));
        Total += amount;
        AmountAdded?.Invoke(this, amount);
    }
}
```

5. When should you use a class instead of a struct?

Use a class for identity, complex behavior, inheritance, large objects, or shared mutable state. Use a struct for small value-like data.

```
public struct Point
{
    public int X { get; set; }
    public int Y { get; set; }
}

public class Customer
{
    public int Id { get; }
    public Customer(int id) => Id = id;
}
```

6. What is the difference between instance and static members?

Instance members belong to a particular object. Static members belong to the type and are shared.

```
public class Counter
{
    public int InstanceCount { get; private set; }
    public static int TotalObjects { get; private set; }
    public Counter() => TotalObjects++;
    public void Increment() => InstanceCount++;
}

var first = new Counter();
var second = new Counter();
first.Increment();
Console.WriteLine(first.InstanceCount);  // 1
Console.WriteLine(second.InstanceCount); // 0
Console.WriteLine(Counter.TotalObjects); // 2
```

7. Can a class contain static and instance constructors?

Yes. A static constructor runs once before the first static access or object creation. An instance constructor runs for every object.

```
public class Configuration
{
    private static string _appName;
    private string _environment;
    static Configuration() => _appName = "Student Portal";
    public Configuration(string environment) => _environment = environment;
    public void Display() => Console.WriteLine($"{_appName} - {_environment}");
}
```

8. What is the purpose of this?

this refers to the current object. It can distinguish fields from parameters, chain constructors, and return the current object.

```
public class Product
{
    private string _name;
    public Product(string name) => this._name = name;
    public Product Rename(string name)
    {
        this._name = name;
        return this;
    }
}
```

9. What is the difference between a class variable and an object reference?

A class variable can hold a reference to an object or null. The object is created separately with new.

```
Book? book = null;
if (book is null)
    Console.WriteLine("No book object exists.");

book = new Book { Title = "C# Fundamentals" };

public class Book
{
    public string Title { get; set; } = string.Empty;
}
```

10. How can you make a class immutable?

Use get-only properties or init-only setters, validate values during construction, and avoid exposing mutable internal state.

```
public sealed class Address
{
    public string City { get; }
    public string Country { get; }
    public Address(string city, string country)
    {
        if (string.IsNullOrWhiteSpace(city))
            throw new ArgumentException("City is required.", nameof(city));
        City = city;
        Country = country;
    }
}
```

11. What is the difference between mutable and immutable objects?

A mutable object can change after creation; an immutable object cannot.

```
public class MutableStudent
{
    public string Name { get; set; } = string.Empty;
}

public sealed class ImmutableStudent
{
    public string Name { get; }
    public ImmutableStudent(string name) => Name = name;
}

var mutable = new MutableStudent { Name = "Ravi" };
mutable.Name = "Arun"; // Allowed
```

12. What are nested classes?

A nested class is declared inside another class. It is useful when the inner type exists only to support the outer type.

```
public class Department
{
    private readonly List<Employee> _employees = new();
    public void AddEmployee(string name) => _employees.Add(new Employee(name));
    private sealed class Employee
    {
        public string Name { get; }
        public Employee(string name) => Name = name;
    }
}
```

13. What is a partial class?

A partial class allows one class to be split across multiple files. The compiler combines all parts into one type.

```
public partial class Customer
{
    public int Id { get; set; }
}

public partial class Customer
{
    public string GetDisplayName() => $"Customer {Id}";
}
```

14. What is a sealed class?

A sealed class cannot be inherited. Use it when the type is complete or further inheritance would be unsafe.

```
public sealed class SecurityToken
{
    public string Value { get; }
    public SecurityToken(string value) => Value = value;
}

// Invalid: public class CustomToken : SecurityToken { }
```

15. What is a static class?

A static class cannot be instantiated or inherited and can contain only static members.

```
public static class TextUtilities
{
```

```
    public static bool IsValid(string? value) =>
        !string.IsNullOrWhiteSpace(value);
    public static string Normalize(string value) =>
        value.Trim().ToUpperInvariant();
}

Console.WriteLine(TextUtilities.Normalize(" hello ")); // HELLO
```

16. Can a class inherit from multiple classes?

No. C# supports single class inheritance, but a class can implement multiple interfaces.

```
public interface ITrainable { void Train(); }
public interface IChargeable { void Charge(); }

public class RobotDog : ITrainable, IChargeable
{
    public void Train() => Console.WriteLine("Training");
    public void Charge() => Console.WriteLine("Charging");
}
```

17. How does object equality differ from reference equality?

Reference equality asks whether two variables refer to the same object. Value equality compares logical contents when implemented.

```
public class Product
{
    public string Code { get; }
    public Product(string code) => Code = code;
}

var first = new Product("P100");
var second = new Product("P100");
Console.WriteLine(first == second); // False
```

18. Why override Equals and GetHashCode together?

Hash-based collections use GetHashCode to locate a bucket and Equals to confirm equality. Equal objects must have equal hash codes.

```
public sealed class Employee
{
    public int Id { get; }
    public Employee(int id) => Id = id;
    public override bool Equals(object? obj) =>
        obj is Employee other && Id == other.Id;
    public override int GetHashCode() => Id.GetHashCode();
}

var set = new HashSet<Employee>
{
    new Employee(101), new Employee(101)
};
Console.WriteLine(set.Count); // 1
```

19. What is the difference between ReferenceEquals, ==, and Equals?

ReferenceEquals checks identity. == uses an overloaded operator when available. Equals checks logical equality according to the type.

```
var first = new Customer { Name = "Ravi" };
var second = new Customer { Name = "Ravi" };
var third = first;

Console.WriteLine(object.ReferenceEquals(first, second)); // False
Console.WriteLine(object.ReferenceEquals(first, third));  // True
Console.WriteLine(first == second);                       // False
Console.WriteLine(first.Equals(third));                   // True

public class Customer
{
    public string Name { get; set; } = string.Empty;
}
```

20. How can you prevent an object from being created in an invalid state?

Validate required values in the constructor and reject invalid input with suitable exceptions.

```
public sealed class BankAccount
```

```
{
    public string AccountNumber { get; }
    public decimal Balance { get; }
    public BankAccount(string number, decimal openingBalance)
    {
        if (string.IsNullOrWhiteSpace(number))
            throw new ArgumentException("Account number is required.");
        if (openingBalance < 0)
            throw new ArgumentOutOfRangeException(nameof(openingBalance));
        AccountNumber = number;
        Balance = openingBalance;
    }
}
```

21. Why expose behavior instead of public fields?

Behavior protects invariants and prevents callers from changing state without validation.

```
public class BankAccount
{
    public decimal Balance { get; private set; }
    public void Deposit(decimal amount)
    {
        if (amount <= 0)
            throw new ArgumentOutOfRangeException(nameof(amount));
        Balance += amount;
    }
}
```

22. How do access modifiers affect class design?

Use the most restrictive access that satisfies the design. private hides implementation; protected supports derived types; internal limits access to an assembly; public exposes an API.

```
public class Example
{
    private int _privateValue;
    protected int _protectedValue;
    internal int _internalValue;
    public int PublicValue { get; set; }
}
```

23. What is object slicing, and does it occur in C#?

C# class assignment uses references, so normal assignment does not slice the object. A base reference can still refer to a derived object.

```
public class Animal
{
    public virtual void Speak() => Console.WriteLine("Animal sound");
}

public class Dog : Animal
{
    public override void Speak() => Console.WriteLine("Bark");
    public void Fetch() => Console.WriteLine("Fetching");
}

Animal animal = new Dog();
animal.Speak(); // Bark
```

24. How would you design a thread-safe class?

Protect shared mutable state with lock, Interlocked, or concurrent collections.

```
public sealed class ThreadSafeCounter
{
    private int _value;
    private readonly object _sync = new();
    public int Value { get { lock (_sync) return _value; } }
    public void Increment()
    {
        lock (_sync) _value++;
    }
}
```

25. Design a BankAccount class that protects its balance.

Keep the balance private, validate all operations, prevent overdrafts, and synchronize access to shared state.

```
public sealed class BankAccount
{
    private readonly object _sync = new();
    private decimal _balance;
    public string AccountNumber { get; }
    public decimal Balance { get { lock (_sync) return _balance; } }
    public BankAccount(string number, decimal opening = 0)
    {
        if (string.IsNullOrWhiteSpace(number))
            throw new ArgumentException("Account number is required.");
        if (opening < 0)
            throw new ArgumentOutOfRangeException(nameof(opening));
        AccountNumber = number;
        _balance = opening;
    }
    public void Deposit(decimal amount)
    {
        if (amount <= 0) throw new ArgumentOutOfRangeException(nameof(amount));
        lock (_sync) _balance += amount;
    }
    public bool Withdraw(decimal amount)
    {
        if (amount <= 0) throw new ArgumentOutOfRangeException(nameof(amount));
        lock (_sync)
        {
            if (amount > _balance) return false;
            _balance -= amount;
            return true;
        }
    }
}
```